Chapter 06

systemd & Containers

Episode 6.01

Episode title: **systemd Fundamentals**

Objective: 2.5

systemd Introduction

Standard init system

Replaces SysVinit and Upstart

Service manager

systemd Architecture and Concepts

PID 1

Units

Dependencies

systemd Unit Types

- Service units
  - Long-running processes
- Timer units
  - Scheduled tasks
- Mount units
  - Filesystem mounts
- Target units
  - System states and runlevels

Managing Services with systemctl

start, stop, restart

enable vs disable

status

Managing systemd Unit States

Reload unit files:

- systemctl daemon-reload

Modify behavior:

- systemctl edit

Control availability:

- mask, unmask

Apply changes:

- reload, restart

Targets and Boot Process

Targets

multi-user.target

graphical.target

Basic systemctl Usage (DEMO)

Inspect units

Start/stop services

Enable at boot

Episode 6.02

Episode title: **systemd Tools & Utilities**

Objective: 2.5

systemd Tools

systemd-analyze

timedatectl

resolvectl

Additional systemd Utilities - Exam

hostnamectl

- Manage system hostname

sysctl

- Kernel parameter tuning

systemd-blame

- Startup timing per unit

systemd-resolved

- DNS resolver service

systemd-analyze

Boot performance analysis

Critical chain

System startup diagnostics

timedatectl

System clock

Time zones

NTP synchronization

resolvectl

DNS resolution

Per-interface settings

Troubleshooting name resolution

Systemd Tools & Utilities (DEMO)

Lorem ipsum dolor sit amet

- Bullet 1
  - Sub-bullet 1

Lorem ipsum dolor sit amet

- Bullet 1
  - Sub-bullet 1

Episode 6.03

Episode title: **Container Basics**

Objective: 2.6

Containers vs Virtual Machines

Shared kernel vs guest OS

Startup time

Resource efficiency

Container Architecture

Namespaces

Control groups (cgroups)

Images and containers

Docker, Podman, and containerd

Docker

Podman

containerd

Container Runtimes (Exam Focus)

runC

- Low-level OCI runtime

Docker

- Full container platform

Podman

- Daemonless container engine

containerd

- Core container runtime

Container Use Cases and Exam Focus

Application deployment

Dev/test environments

Security considerations

Episode 6.04

Episode title: **Container Images**

Objective: 2.6

Container Images Intro

Images are immutable

Ensures consistency

Directly impacts

- Security
- Performance
- Reliability

Container Image Structure and Layers

Layers

Read-only filesystem

Copy-on-write

Bundling Images with Dockerfile

Dockerfile defines image build steps

Common instructions:

- FROM – Base image
- CMD – Default command
- ENTRYPOINT – Fixed executable
- USER – Runtime user

Image Registries and Distribution

Public registries

Private registries

Image tags and versions

Managing Container Images

Pulling images

Listing images

Removing unused images

Container Image Management (DEMO)

Pull images

Inspect images

Remove images safely

Episode 6.05

Episode title: **Container Operations**

Objective: 2.6

Container Lifecycle Management

Start, stop, restart

Foreground vs detached mode

Container states

Container Operations – Exam Coverage

- Run and manage containers
- Start and stop containers
- View logs
- Execute commands inside containers
- Inspect and delete containers

Container Networking

Bridge networking

Port mapping

Network isolation

Container Network Types

Bridge – Default isolated network

Host – Shares host network stack

macvlan / ipvlan

Direct network attachment

Overlay – Multi-host networking

None – No network access

Container Storage and Volumes

Ephemeral storage

Volumes

Bind mounts

Container Volume Operations (Exam)

Create and manage volumes

Map volumes into containers

Prune unused volumes

SELinux context considerations

Overlay storage drivers

Privileged vs Unpriviliged Containers

Privileged containers:

- Full host access
- Reduced isolation -

Unprivileged containers:

- Restricted capabilities
- Improved security

Container Operations (DEMO)

Run containers

Inspect state

Manage networking and volumes